Project Report

(ECE-282: IOT WITH NODE MCU)

LOVELY PROFESSIONAL UNIVERSITY

Topic: Smart IoT Theft Detector

Submitted By:

S. No.	Students Name	Reg. No.	Marks
1.	TARUSH CHAWLA	12304208	
2.	UJJWAL KUMAR RAI	12309624	

Submitted to:

(Dr. Rajat Joshi)

Date of Submission:

FINAL REPORT

Tarush Chawla

School Of Computer Applications

Lovely Professional University

Phagwara, Punjab

Ujjwal Kumar Rai

School Of Computer Applications

Lovely Professional University

Phagwara, Punjab

ABSTRACT

The rising need for accessible and intelligent security solutions has highlighted the limitations of traditional, expensive alarm systems. This project presents the design and implementation of a "Smart IoT Theft Detector with Alarm & Servo Lock," a low-cost, highly effective security system built around the NodeMCU (ESP8266) microcontroller. The system integrates a Passive Infrared (PIR) sensor for motion detection, a servo motor to act as a physical lock, and audible/visual alarms (buzzer and LED). The core innovation lies in its integration with the Blynk IoT platform, enabling users to remotely arm or disarm the system, receive instant intrusion alerts via push notifications, and unlock the servo-controlled mechanism by entering a correct password, all from a smartphone application. This report details the complete project lifecycle, from the initial concept and literature survey to the hardware assembly, circuit design, software development, and final implementation. The result is a robust, scalable, and budget-friendly security solution that demonstrates the power of the Internet of Things (IoT) in modern-day smart home applications.

INTRODUCTION

1.1 PROBLEM STATEMENT

In contemporary society, security remains a paramount concern for homeowners and small businesses alike. Traditional security systems are often prohibitively expensive, involving costly hardware, professional installation, and mandatory monthly subscription fees. Furthermore, these systems lack the flexibility and remote accessibility that modern users demand. A user who forgets to arm their system upon leaving has no recourse, and receiving a simple alert often requires a subscription to a monitoring service. There is a clear and present need for a security solution that is not only affordable and easy to install but also "smart" – providing remote control and instant notifications directly to the user.

1.2 PROPOSED SOLUTION

This project proposes a "Smart IoT Theft Detector" as a modern, low-cost, and flexible alternative to traditional security systems. By leveraging the power of the

Internet of Things (IoT), this system provides robust security features at a fraction of the cost. The system is built using a NodeMCU (ESP8266), a low-cost microcontroller with built-in Wi-Fi capabilities, making it ideal for IoT applications.

The system uses a Passive Infrared (PIR) sensor to detect unauthorized motion. When triggered, it activates a local alarm consisting of a high-decibel active buzzer and a flashing LED. Simultaneously, it sends an instant push notification directly to the user's smartphone via the Blynk IoT platform. The user can remotely arm or disarm the system from anywhere in the world using the Blynk app. As an advanced feature, the system also controls a servo motor, which acts as a physical lock. This lock can be disarmed via a password entered into the app, granting authorized access and bypassing the alarm.

1.3 OBJECTIVES

The primary objectives of this project are as follows:

1. **Low Cost:** To design a fully functional security system using readily available and inexpensive electronic components.
2. **Motion Detection:** To reliably detect motion in a protected area using a PIR sensor.
3. **Remote Control:** To enable the user to arm and disarm the security system remotely using a smartphone application (Blynk).
4. **Instant Alerts:** To send an immediate push notification to the user's smartphone the moment an intrusion is detected.
5. **Local Alarm:** To provide an immediate deterrent to intruders through a loud audible (buzzer) and visual (LED) alarm.
6. **Smart Access:** To implement a password-protected access system via the app that controls a physical servo lock and bypasses the alarm for authorized users.
7. **Scalability:** To create a modular design that can be easily expanded with additional sensors (e.g., door/window sensors, smoke detectors).

LITERATURE SURVEY

The field of home and commercial security has evolved significantly over the past decades. This survey examines existing solutions to contextualize the novelty and utility of the proposed IoT-based system.

Traditional Alarm Systems:
Conventional security systems, such as those provided by companies like ADT, typically involve proprietary hardware, professional installation, and a wired connection to a central monitoring station. While effective, their high initial cost and recurring monthly fees make

them inaccessible to many. Furthermore, they are often "dumb" systems, offering little in the way of remote user interaction beyond a simple keypad.

GSM-Based Alarm Systems: An earlier iteration of "smart" alarms involved using a GSM (Global System for Mobile Communications) module with a microcontroller (like an Arduino Uno) and sensors. When an intrusion was detected, the system would send an SMS (text message) to the owner's phone. While this provided remote notification, it had several drawbacks: the cost of a SIM card and SMS plan, the slow speed of text messages compared to push notifications, and the one-way nature of the communication (i.e., it was difficult to arm/disarm the system remotely via SMS).

Raspberry Pi-Based Systems: More advanced DIY solutions often employ a Raspberry Pi, a powerful single-board computer. These systems can run a full operating system, process video from cameras, and host their own web servers for control. While extremely powerful and flexible, this approach carries a significantly higher cost and complexity. The price of a Raspberry Pi, its power supply, and SD card is many times that of a NodeMCU. It also requires knowledge of Linux and web development, placing it out of reach for many hobbyists.

IoT Platform-Based Systems (Blynk, Cayenne): The emergence of IoT platforms like Blynk has revolutionized the DIY smart home landscape. These platforms provide a pre-built infrastructure (server, app, and library) that handles the complex network communication between the microcontroller and the smartphone. This allows developers to focus purely on the hardware and application logic. Systems using an ESP8266 (like the NodeMCU) and Blynk strike an optimal balance. The hardware is exceptionally low-cost, the Wi-Fi is integrated, and the Blynk platform provides a professional-grade app interface and reliable push notifications for free (on the legacy platform).

This project builds upon the latter approach, combining the cost-effectiveness of the NodeMCU with the user-friendly and powerful features of the Blynk platform. It enhances the basic "PIR-to-notification" concept by adding a state management system (Armed/Disarmed) and a physical access control mechanism (the servo lock), creating a more complete and practical security solution.

METHODOLOGY OF THE PROPOSED SYSTEM

3.1 PRINCIPLE OF OPERATION

The system operates on a core principle of state-based logic managed via the cloud. The NodeMCU microcontroller acts as the central brain, continuously connected to the Blynk IoT server via the local Wi-Fi network.

The system's logic is governed by two key Boolean variables:

1. **systemArmed (bool):** This variable represents the state of the alarm. It is controlled by a button (Virtual Pin V1) in the Blynk app. When true, the system is active and will respond to motion. When false, it is disarmed, and all motion is ignored.
2. **accessGranted (bool):** This variable acts as a temporary bypass. It is controlled by a password input (Virtual Pin V3) in the app. When a user enters the correct password, accessGranted is set to true, the servo lock opens, and the alarm is prevented from sounding even if motion is detected.

The main loop runs two functions: Blynk.run() and timer.run(). The timer.run() function executes the checkMotion() function once every second.

The checkMotion() logic is as follows:

1. **IF** systemArmed is true:
2. Read the PIR_PIN (D5).
3. **IF** the PIR signal is HIGH (motion detected):
4. **IF** accessGranted is false (i.e., an intruder):
5. 1. Trigger the Buzzer (D6).
6. 2. Trigger the LED (D7).
7. 3. Send a push notification via Blynk.logEvent().
8. **ELSE** (if accessGranted is true):
9. - Do nothing. The alarm is bypassed.
10. **ELSE** (if systemArmed is false):
11. Do nothing. The system is disarmed.

This methodology ensures that the system is responsive, flexible, and provides a secure way to manage access without triggering false alarms.

3.2 BLOCK DIAGRAM

The block diagram below illustrates the flow of information and control within the system. The NodeMCU is the central processing unit, managing inputs from both the physical sensor (PIR) and the cloud (Blynk App). It processes this information based on the system's state and controls all outputs (Alarms, Servo, and App Feedback).

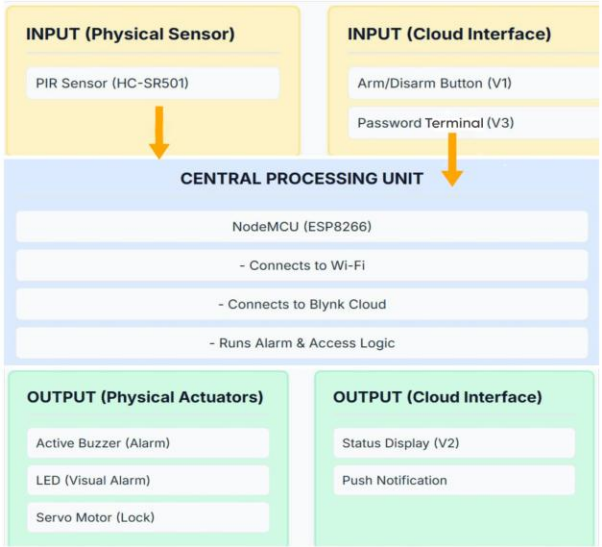

Figure 3.1: Block Diagram of the Smart IoT Theft Detector

3.3 CIRCUIT DIAGRAM AND EXPLANATION

The circuit is designed for simplicity and low power consumption. All components are connected to a central breadboard for power distribution and signal routing to the NodeMCU.

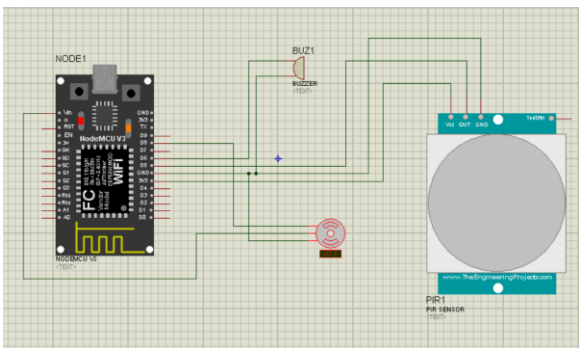

Figure 3.2: Circuit Diagram

Pin Connections:

1. Power (5V): The NodeMCU's Vin pin is connected to the breadboard's positive (red) rail. This rail supplies 5V power to the PIR sensor and the Servo Motor.
2. Ground (GND): The NodeMCU's GND pin is connected to the breadboard's negative (blue) rail. This rail provides a common ground for all components (PIR, LED, Buzzer, and Servo).
3. PIR Sensor (HC-SR501): VCC pin -> Breadboard Positive Rail (5V)
4. GND pin -> Breadboard Negative Rail (GND)
5. OUT pin -> NodeMCU Pin D5 (This is the signal pin that goes HIGH on motion)
6. Active Buzzer: Positive (+) pin -> NodeMCU Pin D6
7. Negative (-) pin -> Breadboard Negative Rail (GND)
8. LED (Light Emitting Diode): Long Leg (Anode, +) -> 220Ω Resistor -> NodeMCU Pin D7
9. Short Leg (Cathode, -) -> Breadboard Negative Rail (GND)
10. Servo Motor (SG90): Red Wire (Power) -> Breadboard Positive Rail (5V)
11. Brown/Black Wire (Ground) -> Breadboard Negative Rail (GND)
12. Orange/Yellow Wire (Signal) -> NodeMCU Pin D8

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

The following hardware components are used in this project:

1. **NodeMCU (ESP8266):** This is the microcontroller at the heart of the project. It is an open-source, low-cost IoT platform. Its key feature is the integrated ESP8266 Wi-Fi chip, which allows it to connect to the

internet and communicate with the Blynk server. It has sufficient GPIO pins to control all the sensors and actuators in this

project.

2. **PIR Sensor (HC-SR501):** A Passive Infrared (PIR) sensor measures infrared light radiating from objects in its field of view. It is designed to detect the heat energy given off by humans (or animals), making it an excellent and low-cost motion detector. It outputs a 3.3V digital HIGH signal when motion is

detected.

3. **Servo Motor (SG90):** A small, lightweight servo motor used to provide a physical locking mechanism. It can rotate 90-180 degrees. In this project, it is moved to 0 degrees ("locked") and 90 degrees ("unlocked") based on

commands from the app.

4. **Active Buzzer:** An active buzzer generates a loud, single-tone sound when a DC voltage is applied. It has an internal oscillator, so it does not require a PWM signal from the microcontroller, simplifying the code. It is used as the audible alarm.

5. **5mm LED:** A standard Light Emitting Diode used as a visual indicator for the alarm.

6. **220Ω Resistor:** A current-limiting resistor used to protect the LED from drawing too much current from the NodeMCU's GPIO pin.

7. **Breadboard and Jumper Wires:** Used for prototyping the circuit without soldering, allowing for easy connection and testing of all components.

4.2 SOFTWARE REQUIREMENTS

The following software platforms and libraries are required:

1. **Arduino IDE (Integrated Development Environment):** This is the open-source software used to write, compile, and upload the C++ based code to the NodeMCU board. The ESP8266 board manager must be added to the IDE to provide support for the NodeMCU.

2. **Blynk (Legacy) App & Platform:** Blynk is an IoT platform that simplifies communication between a microcontroller and a smartphone. The (now Legacy) version provides a simple drag-and-drop interface to build a control dashboard on a smartphone. It provides the widgets (buttons, terminal), server connection, and push notification

functionality for free. An Auth Token from the app is required to link the hardware to the app.

3. **Arduino Libraries: ESP8266WiFi.h**: This library is part of the ESP8266 core and provides the functions needed to connect the NodeMCU to a Wi-Fi network.
4. **BlynkSimpleEsp8266.h**: This is the official Blynk library that handles all communication, authentication, and virtual pin management between the ESP8266 and the Blynk server.
5. **Servo.h**: This library (the ESP8266-compatible version) provides simple functions to control the servo motor, such as attach() and write().

SYSTEM IMPLEMENTATION AND WORKING

5.1 HARDWARE ASSEMBLY

The implementation began with assembling the hardware on a breadboard as per the circuit diagram in Chapter 3.3.

1. The NodeMCU was placed at one end of the breadboard, straddling the central divide.
2. The Vin and GND pins from the NodeMCU were connected to the main positive and negative power rails of the breadboard, respectively. This created a "common ground" and a 5V power rail for the components that require it.
3. The PIR sensor's VCC and GND pins were connected to the 5V and GND rails. Its OUT pin was connected to D5.
4. The SG90 Servo's Red (VCC) and Brown (GND) wires were connected to the 5V and GND rails. Its Orange (Signal) wire was connected to D8.
5. The Active Buzzer's + pin was connected to D6, and its - pin to the GND rail.
6. The LED's short leg (cathode) was connected to the GND rail. Its long leg (anode) was connected to one end of the 220Ω resistor, with the other end of the resistor connected to D7.

This configuration ensures all components are properly powered and share a common ground, which is essential for correct digital signal interpretation.

5.2 BLYNK DASHBOARD SETUP

A crucial part of the implementation is setting up the mobile application interface. This was done using the Blynk (Legacy) app.

1. A new project was created in the Blynk app. The device was set to

"NodeMCU" and connection type to "Wi-Fi".

2. The app generated an Auth Token, which was copied and pasted into the BLYNK_AUTH_TOKEN definition in the Arduino code.
3. The following widgets were added to the dashboard:
4. **Button (V1):** Set to "Switch" mode. This toggles the systemArmed variable. Labels were set to "Armed" (for ON) and "Disarmed" (for OFF).
5. **Labeled Value (V2):** This widget is used for status display. The code (Blynk.virtualWrite(V2, ...)) sends strings like "Armed", "Password Accepted", etc., to this widget.
6. **Terminal (V3):** This widget provides a command-line-like input. The user types the password ("1234") and presses send. The code reads this string from BLYNK_WRITE(V3).
7. **Notification Widget:** This widget requires no setup; adding it to the dashboard simply enables the Blynk.logEvent() function in the code to send push notifications.

5.3 SYSTEM WORKFLOW

The complete operational flow of the system is as follows:

1. Power On & Initialization:

1. The user connects the NodeMCU to a USB power source.
2. The setup() function runs.
3. The Serial Monitor begins (at 115200 baud).
4. All pins (PIR, Buzzer, LED) are set to their modes.
5. The servo is attached to pin D8 and immediately moves to 0 degrees (the "locked" position).
6. The system connects to the Wi-Fi network specified in the code.
7. The system connects to the Blynk server using the Auth Token.
8. The checkMotion() function is set to run every 1000ms (1 second).
9. The system's initial state is set to systemArmed = true, and "Armed" is written to the V2 display.

2. Scenario 1: Intrusion Detected (Alarm)

1. The system is in its "Armed" state.
2. An intruder walks in front of the PIR sensor.
3. The PIR sensor's OUT pin goes HIGH.
4. The next second, checkMotion() runs. It sees systemArmed == true and pirValue == HIGH.
5. It checks accessGranted, which is false.

6. The alarm is triggered: digitalWrite(LED_PIN, HIGH), beepBuzzer(2000, false) runs, and Blynk.logEvent("intrusion_alert", ...) sends a push notification to the user's phone.

3. Scenario 2: Authorized User Access

1. A user approaches the door. They first open the Blynk app.
2. They type the correct password ("1234") into the Terminal (V3) and press Send.
3. The BLYNK_WRITE(V3) function is triggered on the NodeMCU.
4. The code checks if (entered == correctPassword). This is true.
5. accessGranted is set to true.
6. "Password Accepted" is written to V2.
7. myServo.write(90) runs, moving the lock to the "unlocked" position.
8. The user now walks in front of the PIR sensor.
9. checkMotion() runs. It sees systemArmed == true and pirValue == HIGH.
10. It checks accessGranted, which is now true.
11. The if (!accessGranted) block is skipped. No alarm is triggered.

4. Scenario 3: Remote Disarm

1. The user wants to turn the system off.
2. They press the Button (V1) in the Blynk app.
3. The BLYNK_WRITE(V1) function is triggered.
4. systemArmed is set to false.
5. "Disarmed" is written to V2.
6. The servo is moved to 0 degrees (locked) and accessGranted is reset to false as a security precaution.
7. Now, any motion detected by the PIR sensor is ignored by checkMotion() because systemArmed is false.

RESULTS AND DISCUSSION

The system was successfully assembled and tested, and it performed according to all design objectives. The results confirmed that the combination of a NodeMCU, sensors, and the Blynk platform creates a responsive and reliable security system.

1. System Boot and Connection:

Upon powering on, the NodeMCU successfully connected to the specified Wi-Fi network within 5-10 seconds. The Serial Monitor output (Figure 7.1) confirmed a successful Wi-Fi connection and subsequent connection to the Blynk server. The servo motor immediately moved to the 0-degree position, and the

Blynk app updated the status on V2 to "Armed."

2. Blynk App Interface:

The Blynk dashboard (Figure 7.2) provided a simple and intuitive user interface. The Arm/Disarm button (V1) responded instantly, and the Labeled Value display (V2) correctly reflected the system's current state ("Armed", "Disarmed", "Password Accepted", "Wrong Password").

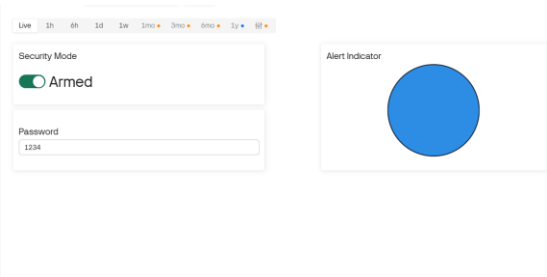

Figure 7.1: Blynk App Dashboard

3. System Functionality:

1. **Intrusion Test:** While the system was "Armed," waving a hand in front of the PIR sensor resulted in an immediate response: the LED and Buzzer activated for 2 seconds, and a push notification (Figure 7.3) was received on the smartphone within 1-2 seconds.
2. **Password Access Test:** Entering "1234" into the V3 terminal correctly set the accessGranted flag, moved the servo to 90 degrees, and updated the status to "Password Accepted." Subsequent motion in front of the PIR sensor did not trigger the alarm.
3. **Wrong Password Test:** Entering "1111" updated the status to "Wrong Password" and did not move the servo, as expected.

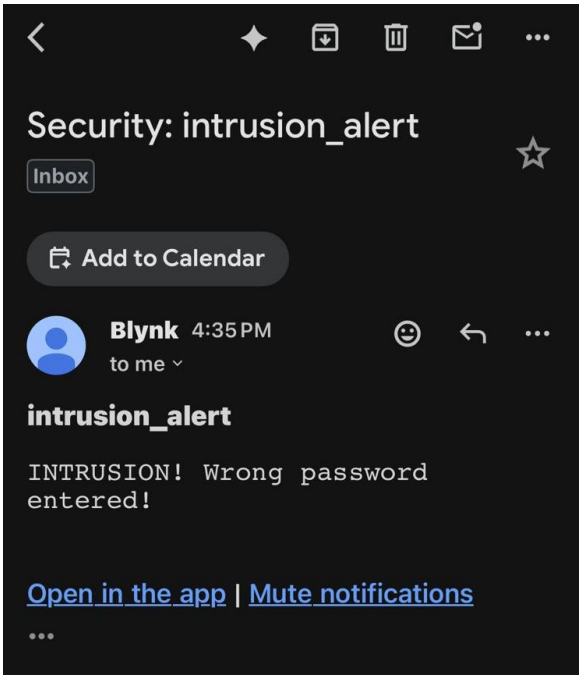

Figure 7.3: Intrusion Alert Notification

Discussion of Challenges:

One minor challenge encountered was a false alarm immediately upon boot-up. This is due to the PIR sensor's "warm-up" period, during which its output pin goes HIGH for several seconds. Since the code starts checking for motion after 1 second, it interprets this as an intrusion. A simple fix would be to add a delay(30000) (30-second delay) in the setup() function to allow the sensor to calibrate before the main loop begins. Another challenge is

the high current draw of the servo motor, which can sometimes cause the NodeMCU to reset if powered from a weak USB port. For a permanent installation, a dedicated 5V 2A external power supply would be recommended.

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

This project successfully demonstrates the design, development, and implementation of a Smart IoT Theft Detector. By integrating a NodeMCU ESP8266 with a PIR sensor, a servo motor, and the Blynk IoT platform, we have created a security system that is not only highly functional but also extremely low-cost and accessible. The system meets all its stated objectives: it provides reliable motion detection, instant push notifications, remote arm/disarm capabilities, and a password-protected physical lock.

The project highlights the transformative potential of IoT technology in the home security domain. It proves that a robust, smart, and interactive security system can be built by hobbyists and students, democratizing access to technology that was once reserved for expensive, professionally-installed systems. The final prototype is a practical and effective solution for securing a room, desk, or personal cabinet.

8.2 FUTURE SCOPE

While the current system is fully functional, its modular design allows for numerous potential upgrades and enhancements. Future work on this project could include:

1. **Additional Sensors:** Integrating magnetic reed switches (for doors and windows) or vibration sensors to provide different types of intrusion detection.
2. **Camera Integration:** Upgrading the microcontroller to an ESP32-CAM, which could be programmed to take a photograph of the intruder when the PIR sensor is triggered and email it to the user.
3. **Blynk 2.0 Migration:** Migrating the project from Blynk (Legacy) to the new Blynk 2.0 platform, which offers more advanced features, data logging, and long-term support.
4. **Battery Backup:** Adding a battery pack (e.g., using a LiPo battery with a charging module) to ensure the system remains operational during a power outage.
5. **Two-Factor Authentication:** Implementing a more secure access protocol, such as requiring both the password and an RFID tag scan to disarm the system.

REFERENCES

1. Blynk (Legacy) Documentation. (n.d.). Retrieved from https://docs.blynk.io/en/legacy
2. NodeMCU ESP8266 Datasheet. (n.d.). Espressif Systems. Retrieved from https://www.espressif.com/en/products/socs/esp8266
3. Arduino IDE. (n.d.). Retrieved from https://www.arduino.cc/en/software
4. HC-SR501 PIR Sensor Datasheet. (n.d.). Retrieved from https://www.mpja.com/download/31227mp.pdf
5. TowerPro SG90 Servo Motor Datasheet. (n.d.). Retrieved from http://www.ee.ic.ac.uk/pcheung/teaching/DE1-1/docs/SG90-datasheet.pdf
6. Monk, S. (2016). Programming Arduino: Getting Started with Sketches. McGraw-Hill Education.